



**UNIVERSITY OF FRONTIER TECHNOLOGY,
BANGLADESH**

Department of Cyber Security Engineering

Project Report

Project Title : SecureGov Framework: An Integrated Security, Governance and Risk Analysis Platform

Course title : Security Governance, Risk Management and Compliance Sessional

Course code : SEC 116

Submitted by : Faria Yasmin Jui (2404036) Momtasin Tamann (2404012) Anika Akter nira (2404034) Level : 01 Term : 02 Department of Cyber Security Engineering, UFTB	Submitted to: Rakib Hossen Assistant Professor, Department of Cyber Security Engineering, UFTB
---	--

SecureGov Framework : An Integrated Security, Governance & Risk Analysis Platform

Introduction

The SecureGov Framework is a unified platform that brings security, governance, and risk management into one place for government organizations. Usually, these things are handled separately, which creates huge blind spots and makes it hard to track cyber threats quickly. SecureGov fixes this by linking network scanning, vulnerability detection, and exploit testing into a single workflow. Built inside Kali Linux, it uses popular open-source tools to monitor network health in real time. The main goal is simple: take complicated system logs and turn them into clear, actionable compliance updates for decision-makers.

Objectives

- Real-time security monitoring and risk visibility.
- Automate compliance assessment and report generation.
- Detect and prioritize security vulnerabilities.
- Centralize security findings into a unified dashboard.
- Reduce manual work and human errors.
- Enable proactive risk identification and mitigation.
- Support vulnerability lifecycle management.
- Generate comprehensive executive and technical reports.

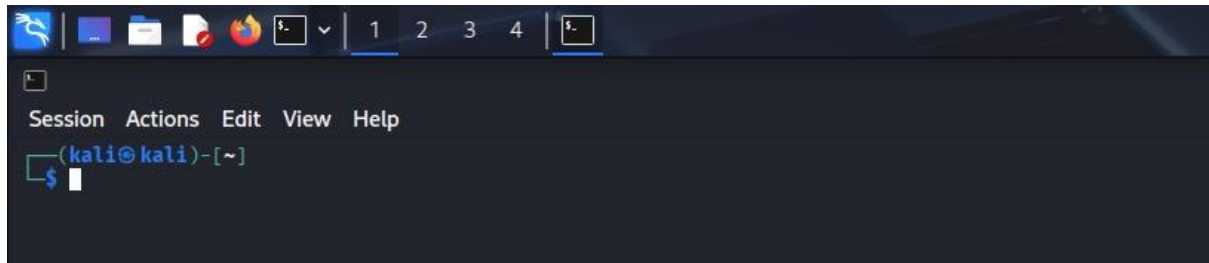
Tools

- **Kali Linux:** Primary operating system used for the entire security testing environment.
- **Nmap:** Asset discovery and network mapping tool.
- **OpenVAS:** Vulnerability assessment and risk detection tool.
- **Wireshark:** Packet inspection and traffic analysis utility.
- **Metasploit:** Vulnerability and controlled exploitation testing framework.

Implementation Steps:

Kali Linux

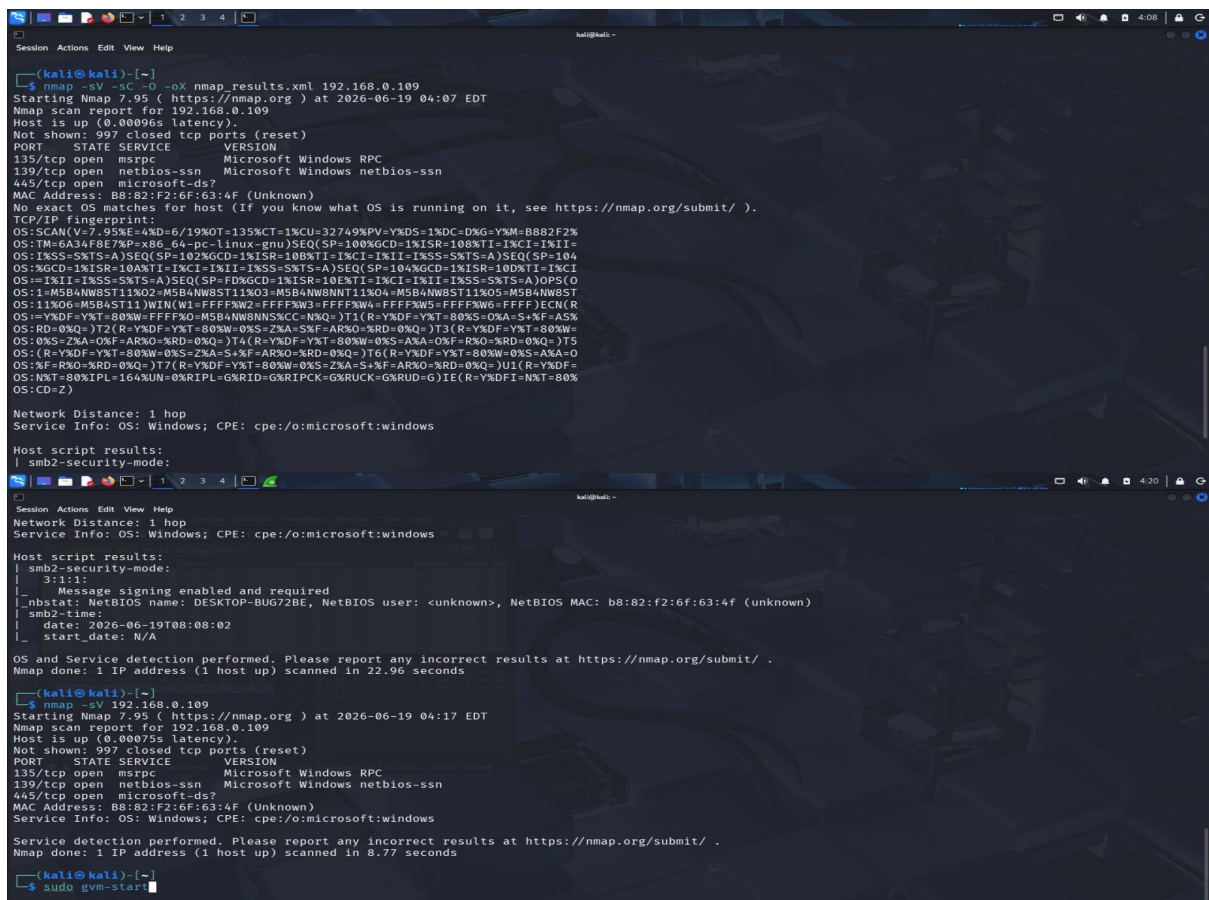
Open Kali Linux Terminal First, run your Kali Linux OS. Open the terminal and ensure your system is updated.

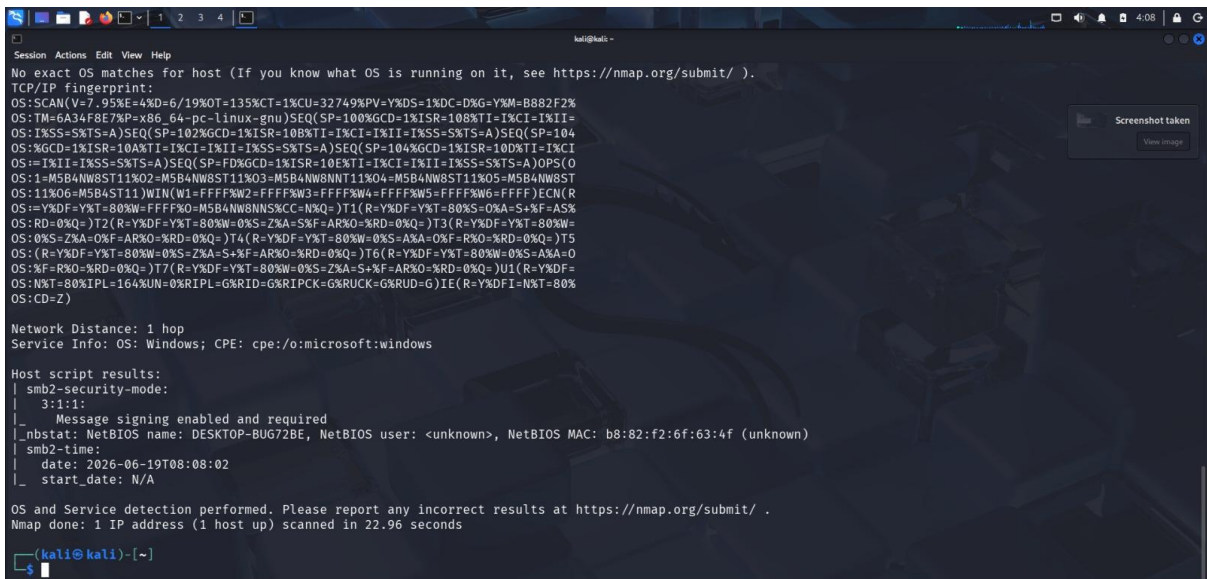


Reconnaissance and Network Scanning (Nmap)

To identify whether the target host (**192.168.0.109**) is online and determine which ports and services are running, an aggressive Nmap scan was performed.

```
nmap -sV -sC -O -oX nmap_results.xml 192.168.0.109
```



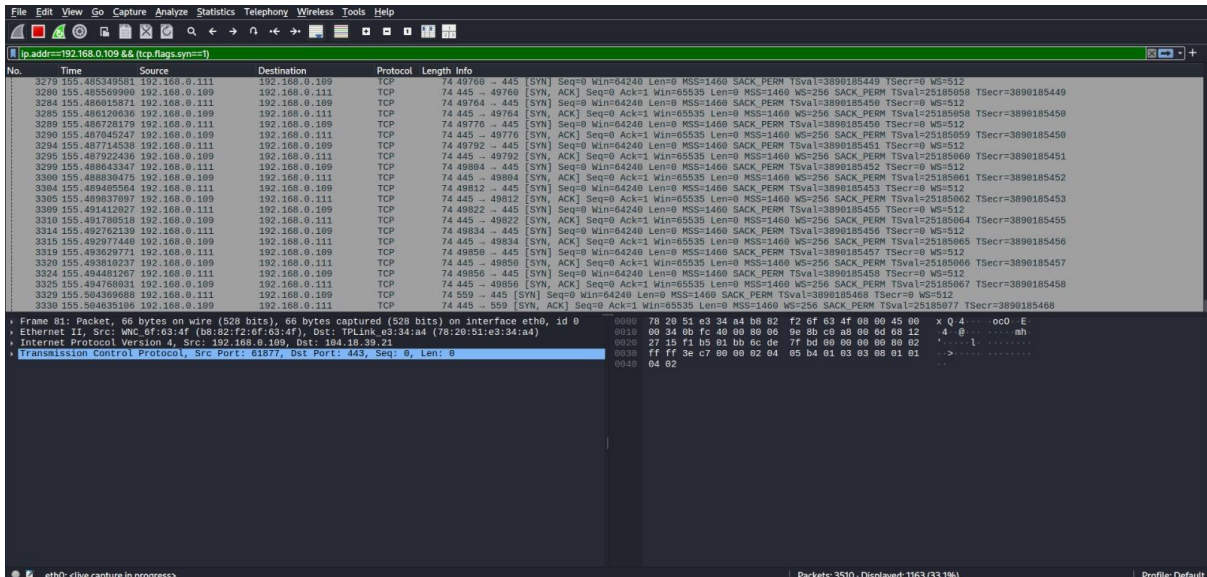


Traffic Capture & Handshake Analysis (Wireshark)

While the Nmap scan was running, Wireshark was used to monitor the network traffic and observe the TCP Three-Way Handshake between the scanner and the target host.

Wireshark Display Filter:

ip.addr == 192.168.0.109 && (tcp.flags.syn == 1)

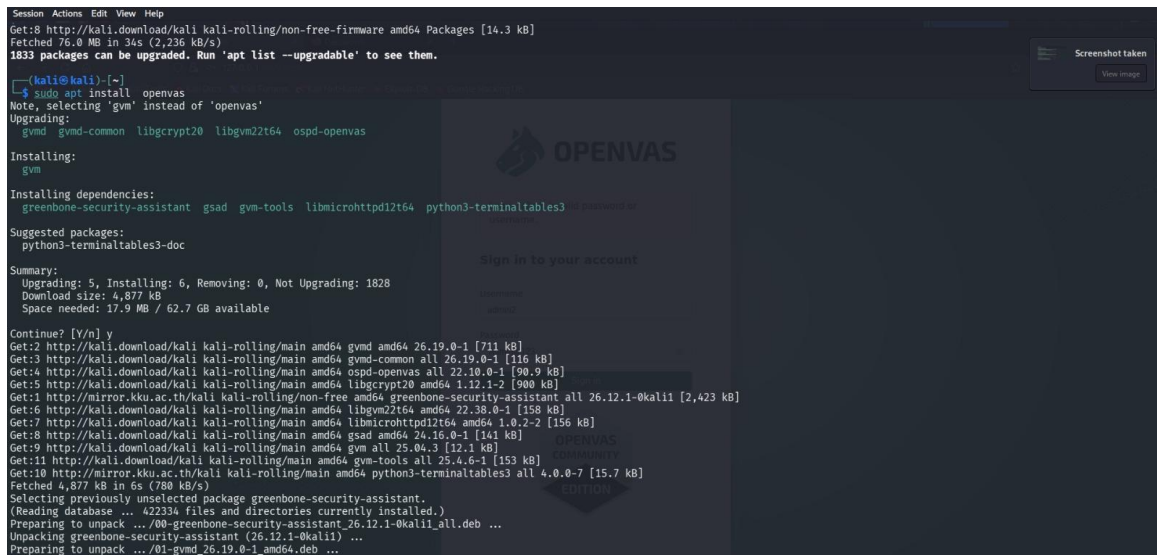


Automated Vulnerability Assessment (OpenVAS / GVM)

Step-1: Package Installation

The installation of **OpenVAS (Greenbone Vulnerability Manager - GVM)** begins by executing the following command in the terminal:

```
sudo apt install openvas
```

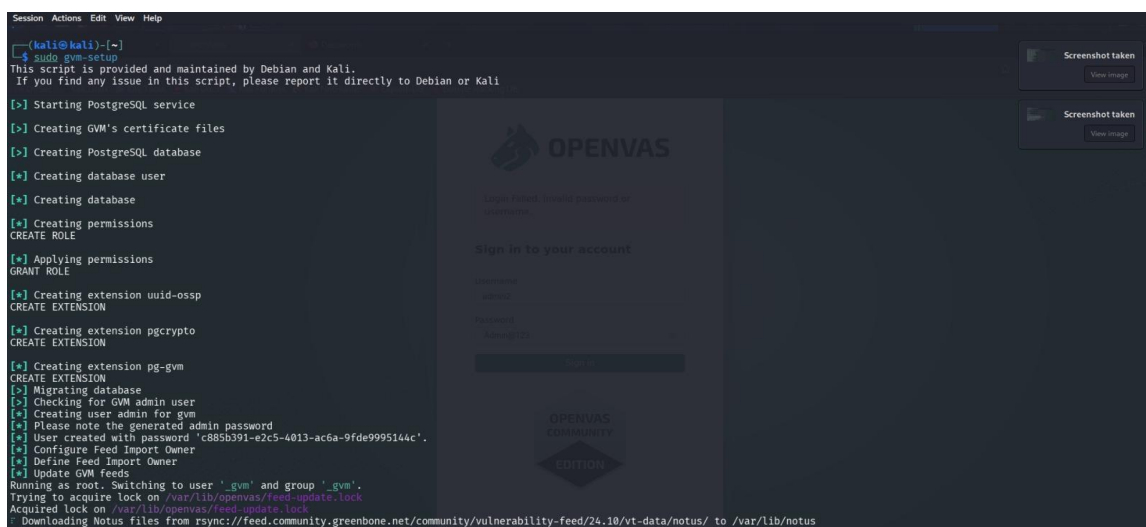


The screenshot shows a terminal window on the left and the OpenVAS web interface on the right. The terminal output shows the command `sudo apt install openvas` being executed, which selects 'gvm' instead of 'openvas'. It lists the packages to be upgraded (gvm, gvm-common, libgcrpt20, libgvm22t64, ospd-openvas) and the dependencies (greenbone-security-assistant, gsad, gvm-tools, libmicrohttpd12t64, python3-terminaltables3). The summary shows that 5 packages will be upgraded, 6 will be installed, and 0 will be removed, with a total download size of 4,877 kB. The user confirms the installation with 'y'. The terminal then shows the progress of downloading and installing the packages. The web interface on the right shows the OpenVAS logo, a sign-in prompt, and a download button.

Step-2: Initial Setup & User Creation

After the installation is complete, the following command is executed to perform the initial setup of GVM:

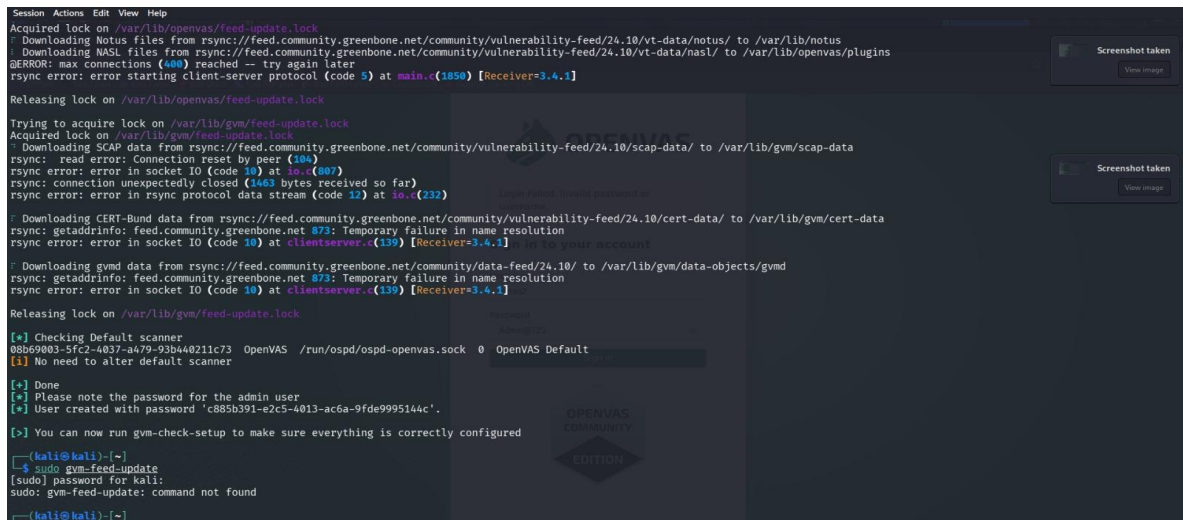
```
sudo gvm-setup
```



The screenshot shows a terminal window on the left and the OpenVAS web interface on the right. The terminal output shows the command `sudo gvm-setup` being executed. It starts by starting the PostgreSQL service, creating GVM's certificate files, creating the PostgreSQL database, creating a database user, creating the database, creating permissions, creating the role, applying permissions, creating the extension uuid-osp, creating the extension pgcrypto, creating the extension pg-gvm, migrating the database, checking for the GVM admin user, creating the user admin for gvm, and creating the user created with password 'c885b391-e2c5-4013-ac6a-9fde9995144c'. It then defines the feed import owner, updates the GVM feeds, and switches to the user 'gvm' and group 'gvm'. Finally, it acquires the lock on the feed update lock and downloads the Notus files from rsync://feed.community.greenbone.net/community/vulnerability-feed/24.10/vt-data/notus/ to /var/lib/notus. The web interface on the right shows the OpenVAS logo, a sign-in prompt, and a download button.

Step-3: Feed Update Errors & Troubleshooting

This stage shows network synchronization errors (rsync connection resets) during the vulnerability data stream update process. It also displays a "command not found" error when attempting an invalid sudo gvm-feed-update command.



```
Session Actions Edit View Help
Acquired lock on /var/lib/openvas/feed-update.lock
Downloading Notus files from rsync://feed.community.greenbone.net/community/vulnerability-feed/24.10/vt-data/notus/ to /var/lib/notus
Downloading NASL files from rsync://feed.community.greenbone.net/community/vulnerability-feed/24.10/vt-data/nasl/ to /var/lib/openvas/plugins
ERROR: max connections (400) reached - try again later
rsync error: error starting client-server protocol (code 5) at main.c(1850) [Receiver=3.4.1]
Releasing lock on /var/lib/openvas/feed-update.lock
Trying to acquire lock on /var/lib/gvm/feed-update.lock
Acquired lock on /var/lib/gvm/feed-update.lock
Downloading SCAP data from rsync://feed.community.greenbone.net/community/vulnerability-feed/24.10/scap-data/ to /var/lib/gvm/scap-data
rsync: read error: Connection reset by peer (104)
rsync error: error in socket IO (code 10) at io.c(807)
rsync: connection unexpectedly closed (1463 bytes received so far)
rsync error: error in rsync protocol data stream (code 12) at io.c(232)
Downloading CERT-Bund data from rsync://feed.community.greenbone.net/community/vulnerability-feed/24.10/cert-data/ to /var/lib/gvm/cert-data
rsync: getaddrinfo: feed.community.greenbone.net 873: Temporary failure in name resolution
rsync error: error in socket IO (code 10) at clientserver.c(139) [Receiver=3.4.1]
Downloading gvm data from rsync://feed.community.greenbone.net/community/data-feed/24.10/ to /var/lib/gvm/data-objects/gvmd
rsync: getaddrinfo: feed.community.greenbone.net 873: Temporary failure in name resolution
rsync error: error in socket IO (code 10) at clientserver.c(139) [Receiver=3.4.1]
Releasing lock on /var/lib/gvm/feed-update.lock
[+] Checking Default scanner
08b69003-5fc2-4037-a479-93b440211c73 OpenVAS /run/ospd/ospd-openvas.sock 0 OpenVAS Default
[+] No need to alter default scanner
[+] Done
[+] Please note the password for the admin user
[+] User created with password 'c885b391-e2c5-4013-ac6a-9fde9995144c'.
[+] You can now run gvm-check-setup to make sure everything is correctly configured
(kali@kali)-[~]
$ sudo gvm-feed-update
[sudo] password for kali:
sudo: gvm-feed-update: command not found
(kali@kali)-[~]
```

Step-4: Feed Synchronization

Before performing any vulnerability assessment, the vulnerability database and security signatures must be updated using the following command:

sudo greenbone-feed-sync



```
(kali@kali)-[~]
$ sudo greenbone-feed-sync
Running as root. Switching to user 'gvm' and group 'gvm'.
Trying to acquire lock on /var/lib/openvas/feed-update.lock
Acquired lock on /var/lib/openvas/feed-update.lock
Downloading Notus files from rsync://feed.community.greenbone.net/community/vulnerability-feed/24.10/vt-data/notus/ to /var/lib/notus
Downloading NASL files from rsync://feed.community.greenbone.net/community/vulnerability-feed/24.10/vt-data/nasl/ to /var/lib/openvas/plugins
Releasing lock on /var/lib/openvas/feed-update.lock
Trying to acquire lock on /var/lib/gvm/feed-update.lock
Acquired lock on /var/lib/gvm/feed-update.lock
Downloading SCAP data from rsync://feed.community.greenbone.net/community/vulnerability-feed/24.10/scap-data/ to /var/lib/gvm/scap-data
Downloading CERT-Bund data from rsync://feed.community.greenbone.net/community/vulnerability-feed/24.10/cert-data/ to /var/lib/gvm/cert-data
Downloading gvmd data from rsync://feed.community.greenbone.net/community/data-feed/24.10/ to /var/lib/gvm/data-objects/gvmd
Releasing lock on /var/lib/gvm/feed-update.lock
```

Step-5: Setup Verification

After the synchronization process is completed, the following command is executed to verify that all required services and configurations are functioning correctly:

sudo gvm-check-setup

```
(kali@kali)-[~]
$ sudo gvm-check-setup
[sudo] password for kali:
gvm-check-setup 25.04.0
This script is provided and maintained by Debian and Kali.
Test completeness and readiness of GVM-25.04.0
Step 1: Checking OpenVAS (Scanner)...
```

```
Session Actions Edit View Help
● ospd-openvas.service - OSPd Wrapper for the OpenVAS Scanner (ospd-openvas)
   Loaded: loaded (/usr/lib/systemd/system/ospd-openvas.service; disabled; preset: disabled)
   Active: active (running) since Fri 2026-06-19 04:20:10 EDT; 16s ago
     Invocation: 0ca2e7ff30074f668155d57097b235c3
       Docs: man:ospd-openvas(8)
            man:openvas(8)
    Process: 19423 ExecStart=/usr/bin/ospd-openvas --config /etc/gvm/ospd-openvas.conf --log-config /etc/gvm/ospd-logging.conf (code=exited, status=0/SUCCESS)
   Main PID: 19460 (ospd-openvas)
      Tasks: 5 (limit: 4454)
     Memory: 324.1M (peak: 370M)
        CPU: 5.127s
    CGroup: /system.slice/ospd-openvas.service
            └─19460 /usr/bin/python3 /usr/bin/ospd-openvas --config /etc/gvm/ospd-openvas.conf --log-config /etc/gvm/ospd-logging.conf
              └─19462 /usr/bin/python3 /usr/bin/ospd-openvas --config /etc/gvm/ospd-openvas.conf --log-config /etc/gvm/ospd-logging.conf

Jun 19 04:20:08 kali systemd[1]: Starting ospd-openvas.service - OSPd Wrapper for the OpenVAS Scanner (ospd-openvas) ...
Jun 19 04:20:10 kali systemd[1]: Started ospd-openvas.service - OSPd Wrapper for the OpenVAS Scanner (ospd-openvas).

[>] Opening Web UI (https://127.0.0.1:9392) in: 5 ... 4 ... 3 ... 2 ... 1 ...
```

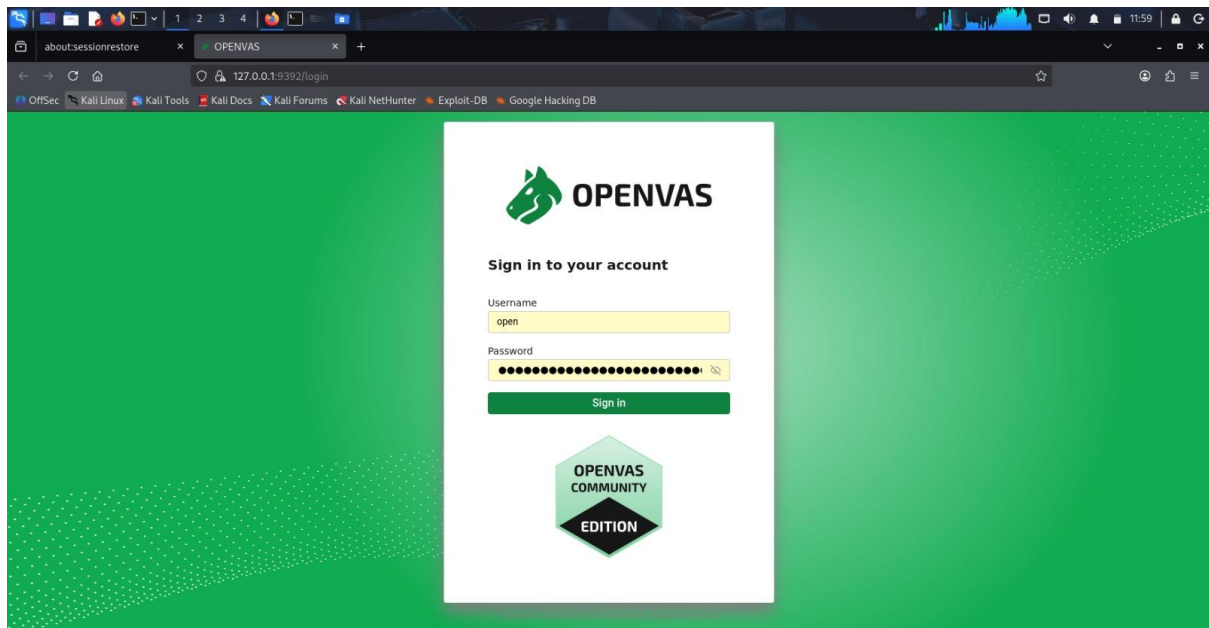
sudo runuser -u _gvm --gvmd --create-user=<username> --new-password=<password>

```
(kali@kali)-[~]
$ sudo runuser -u _gvm --gvmd --create-user=openvas --new-password=new
User created with password '9c8ab181-500c-413b-90ac-ed18537f32f7'.
```

Step-6: Successful Web Portal Login

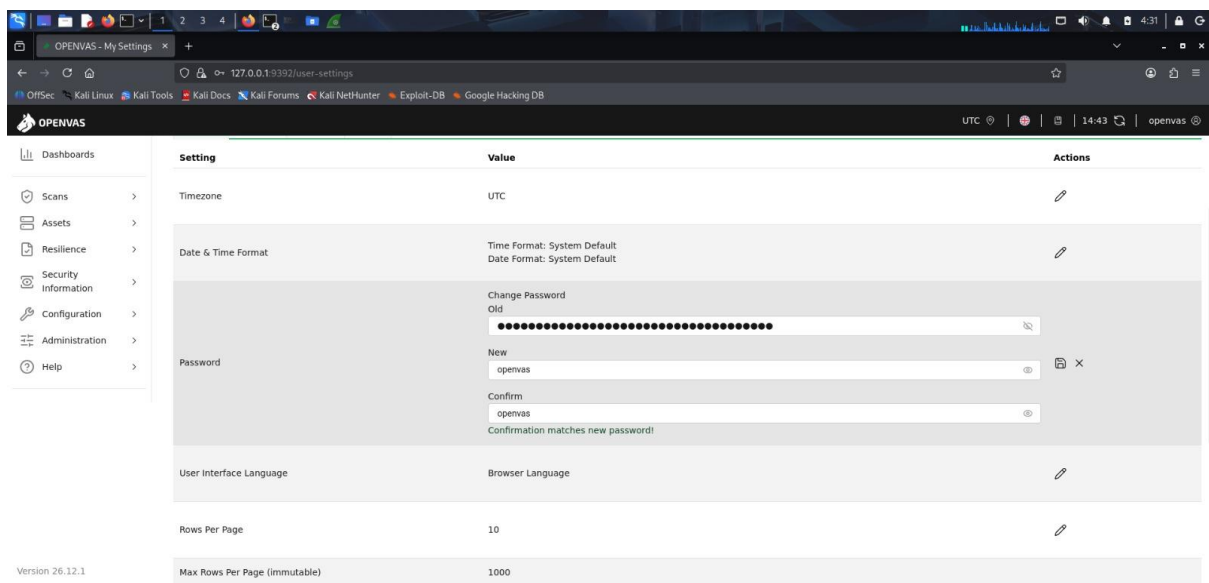
Once the setup and feed synchronization are completed successfully, the web management portal can be accessed by opening a web browser and navigating to:

<https://localhost:9392>



Step-7: Password Modification (User Settings)

The **My Settings** page accessed immediately after logging in to change the complex, auto-generated default password to a preferred custom password.

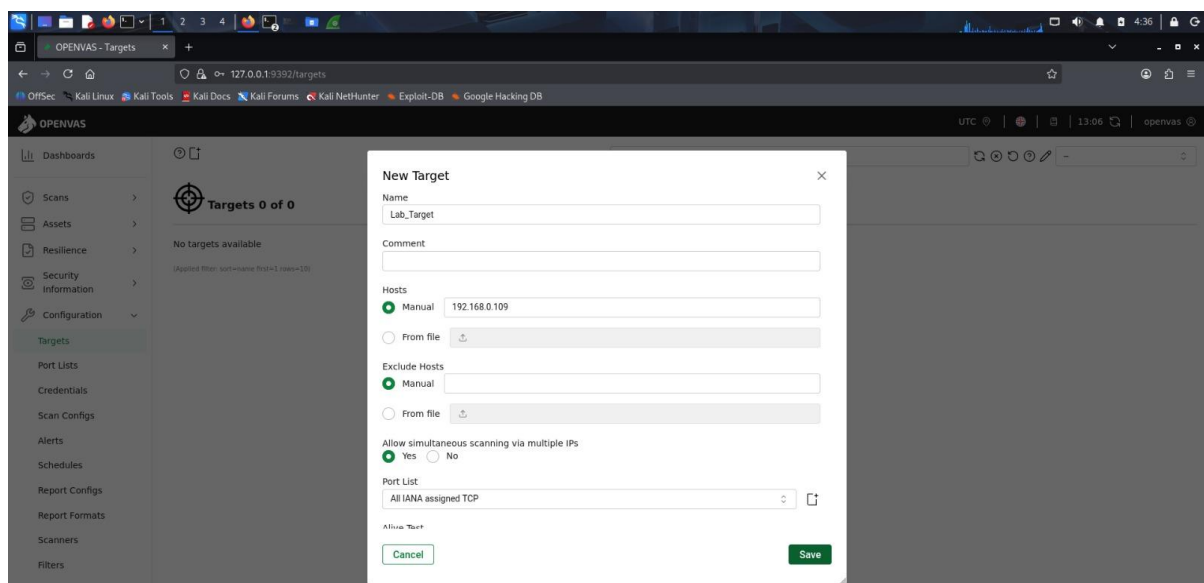


Step-8: Scan Target Configuration

Before launching a vulnerability scan, the target host must be configured within the GVM dashboard.

Configuration Steps:

- Navigate to **Configuration** → **Targets**.
- Create a new target.
- Enter the following information:
 - **Target Name:** Lab_Target
 - **Host IP Address:** 192.168.0.109
- Save the target configuration.

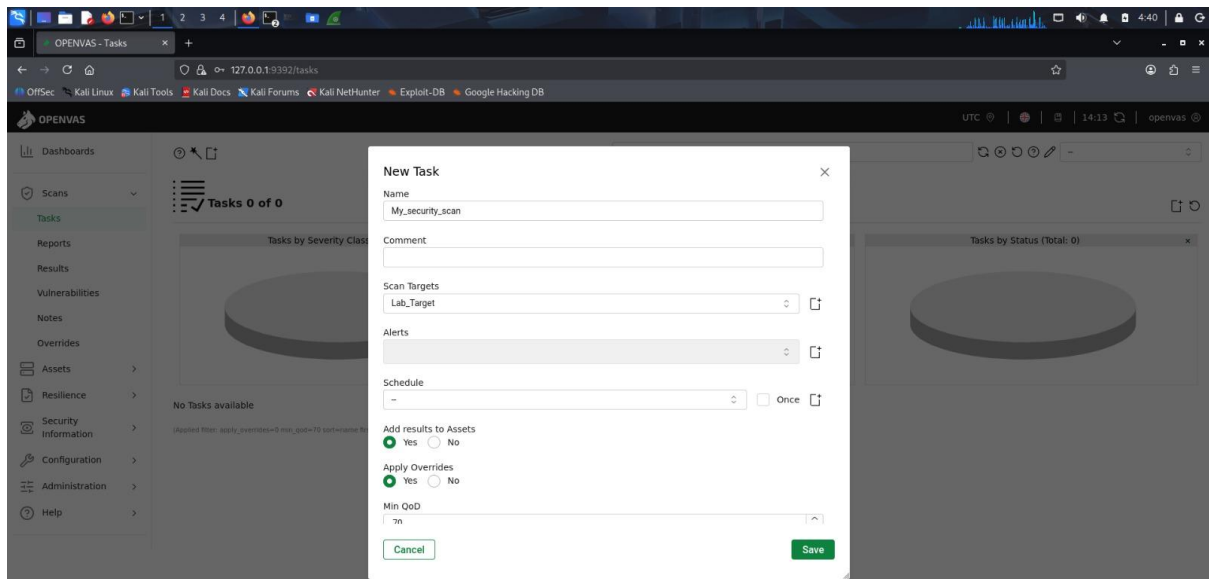


Step-9: Creating a New Scan Task

After configuring the target, a new scan task is created to perform the vulnerability assessment.

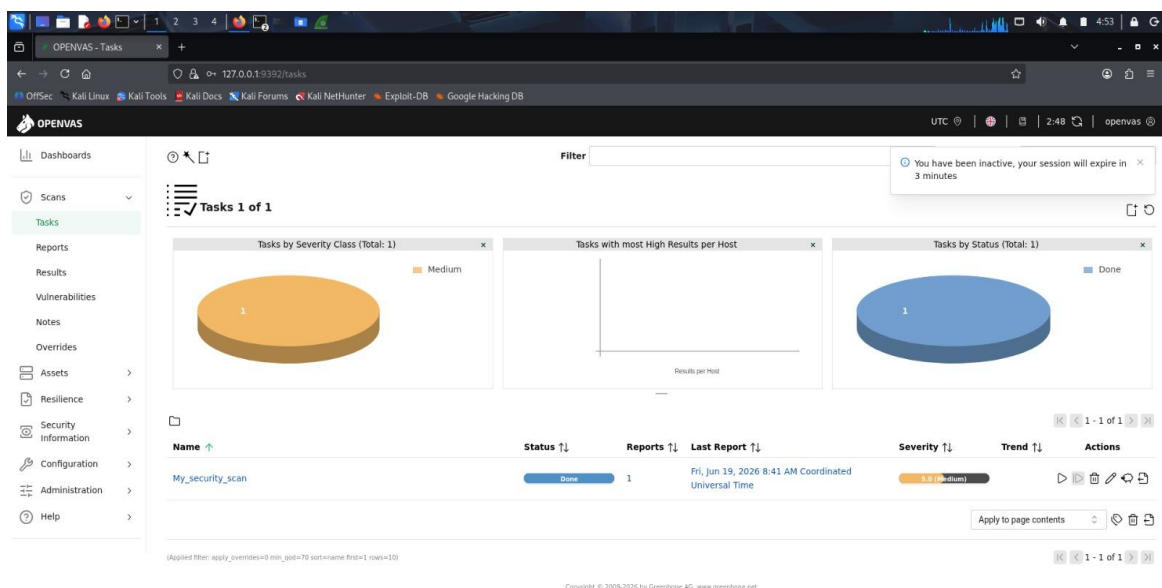
Configuration Steps:

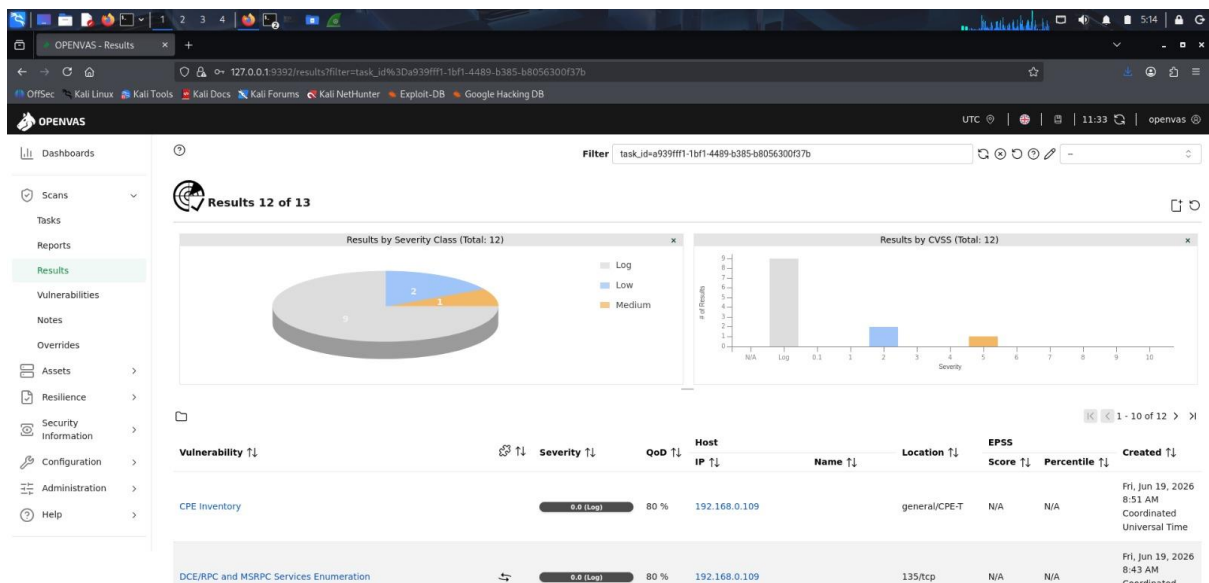
- Navigate to **Scans** → **Tasks**.
- Select **New Task**.
- Configure the following:
 - **Task Name:** My_security_scan
 - **Scan Target:** Lab_Target
- Save the task.
- Start the vulnerability scan by clicking the **Play (Start)** button.



Step-10: Analyzing Scan Results and Generating Reports

After the vulnerability scan completes successfully, the task status changes to **Done**, indicating that the assessment has finished.





```
Session Actions Edit View Help
● ospd-openvas.service - OSPd Wrapper for the OpenVAS Scanner (ospd-openvas)
  Loaded: loaded (/usr/lib/systemd/system/ospd-openvas.service; disabled; preset: disabled)
  Active: active (running) since Fri 2026-06-19 04:20:10 EDT; 16s ago
  Invocation: 0ca2e7ff30074f668155d57097b235c3
  Docs: man:ospd-openvas(8)
        man:openvas(8)
  Process: 19423 ExecStart=/usr/bin/ospd-openvas --config /etc/gvm/ospd-openvas.conf --log-config /etc/gvm/ospd-logging.conf (code=exited, status=0/SUCCESS)
  Main PID: 19460 (ospd-openvas)
  Tasks: 5 (limit: 4454)
  Memory: 324.1M (peak: 370M)
  CPU: 5.127s
  CGroup: /system.slice/ospd-openvas.service
          └─19460 /usr/bin/python3 /usr/bin/ospd-openvas --config /etc/gvm/ospd-openvas.conf --log-config /etc/gvm/ospd-logging.conf
            └─19462 /usr/bin/python3 /usr/bin/ospd-openvas --config /etc/gvm/ospd-openvas.conf --log-config /etc/gvm/ospd-logging.conf

Jun 19 04:20:08 kali systemd[1]: Starting ospd-openvas.service - OSPd Wrapper for the OpenVAS Scanner (ospd-openvas)...
Jun 19 04:20:10 kali systemd[1]: Started ospd-openvas.service - OSPd Wrapper for the OpenVAS Scanner (ospd-openvas).

[>] Opening Web UI (https://127.0.0.1:9392) in: 5 ... 4 ... 3 ... 2 ... 1 ...

(kali@kali)-[~]
└─$ sudo runuser -u _gvm -- gvmc --create-user=admin --new-password=new
User exists already.

(kali@kali)-[~]
└─$ sudo runuser -u _gvm -- gvmc --create-user=openvas --new-password=new
User created with password '9c8ab181-500c-413b-90ac-ed18537f32f7'.

(kali@kali)-[~]
└─$ msfconsole
Metasploit tip: You can upgrade a shell to a Meterpreter session on many
platforms using sessions -u <session_id>
[*] Starting the Metasploit Framework console ... /
```

Vulnerability Verification (Metasploit Framework)

Step-1: Starting Metasploit Framework Console

The vulnerabilities and service information identified by OpenVAS were verified using Metasploit's auxiliary scanning modules.

```
(kali@kali)-[~]
└─$ msfconsole
Metasploit tip: You can upgrade a shell to a Meterpreter session on many
platforms using sessions -u <session_id>
[*] Starting the Metasploit Framework console ... /
```

Step-2: Download DefectDojo and Start Docker (Terminal)

In the first image, the **DefectDojo** repository is cloned from GitHub, and the application is started using **Docker Compose**.

Clone the DefectDojo repository from GitHub

git clone <https://github.com/DefectDojo/django-DefectDojo.git>

```
(kali@kali)-[~]
$ git clone "https://github.com/DefectDojo/django-DefectDojo.git"
Cloning into 'django-DefectDojo' ...
remote: Enumerating objects: 311628, done.
remote: Counting objects: 100% (966/966), done.
remote: Compressing objects: 100% (337/337), done.
Receiving objects: 100% (311628/311628), 332.12 MiB | 1.91 MiB/s, done.
remote: Total 311628 (delta 787), reused 631 (delta 629), pack-reused 310662 (from 3)
Resolving deltas: 100% (180116/180116), done.
Updating files: 100% (4759/4759), done.
```

Step-3: Navigate to the project directory

cd django-DefectDojo

```
(kali@kali)-[~]
$ cd django-DefectDojo

(kali@kali)-[~/django-DefectDojo]
$
```

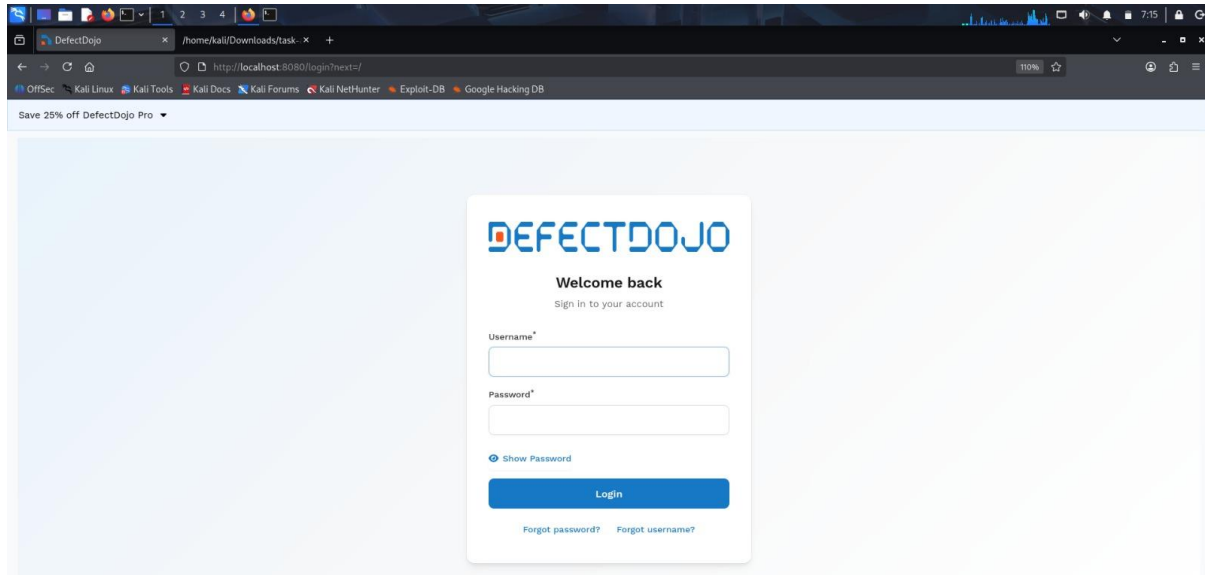
Step-4: Start the services in the background using Docker Compose

sudo docker-compose up -d

```
(kali@kali)-[~/django-DefectDojo]
$ sudo docker-compose up -d
[sudo] password for kali:
[+] Running 0/7
  ⚙️ celeryworker Pulling
  ⚙️ initializer Pulling
  ⚙️ postgres Pulling
  ⚙️ valkey Pulling
  ⚙️ nginx Pulling
  ⚙️ uwsgi Pulling
  ⚙️ celerybeat Pulling
```

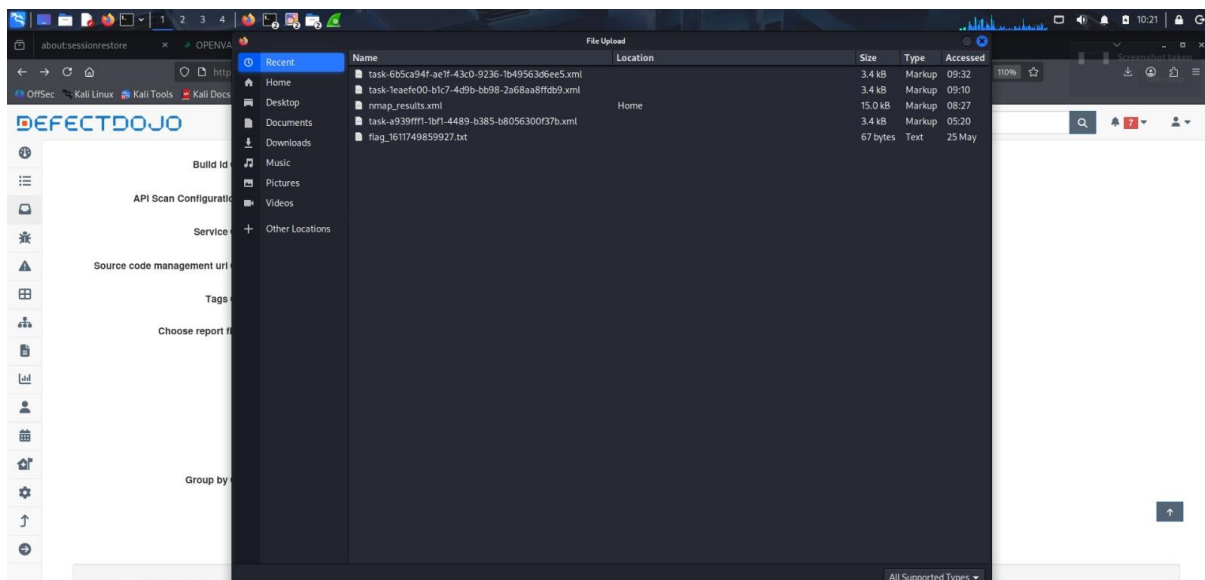
Step-5: Log in to the Dashboard (Browser Login)

In the second image, after the Docker containers have started successfully, open <http://localhost:8080> in a web browser to access the DefectDojo login page. Then, log in using the assigned **Username** and **Password**.



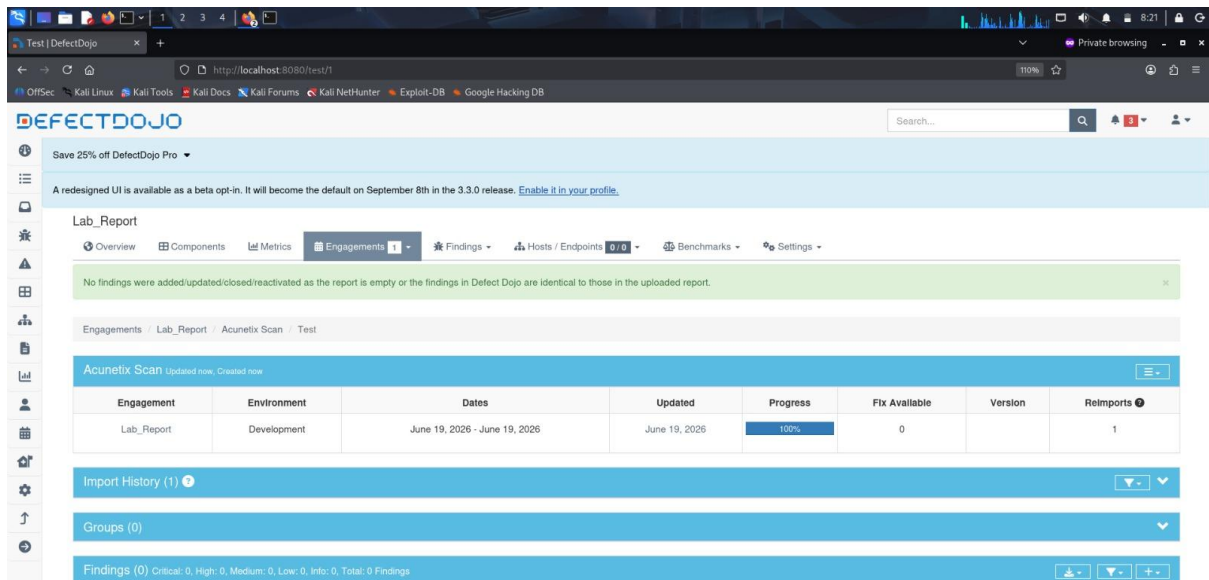
Step-6: Report File Upload

After logging in, this file selection window appears, allowing you to upload the vulnerability report file (such as .xml outputs from Nmap, OpenVAS, or Acunetix).



Step-7: Initial Engagement & Import Status Verification

This step shows the status of an active engagement (labeled as an *Acunetix Scan* here). A green banner notice confirms that the file was processed, though no new findings were added because the report was either empty or identical to a previous upload.



DEFECTDOJO

Save 25% off DefectDojo Pro

A redesigned UI is available as a beta opt-in. It will become the default on September 8th in the 3.3.0 release. [Enable it in your profile.](#)

Lab_Report

Overview Components Metrics Engagements 1 Findings Hosts / Endpoints 0/0 Benchmarks Settings

No findings were added/updated/closed/reactivated as the report is empty or the findings in Defect Dojo are identical to those in the uploaded report.

Engagements Lab_Report Acunetix Scan Test

Acunetix Scan Updated now, Created now

Engagement	Environment	Dates	Updated	Progress	Fix Available	Version	Reimports
Lab_Report	Development	June 19, 2026 - June 19, 2026	June 19, 2026	100%	0		1

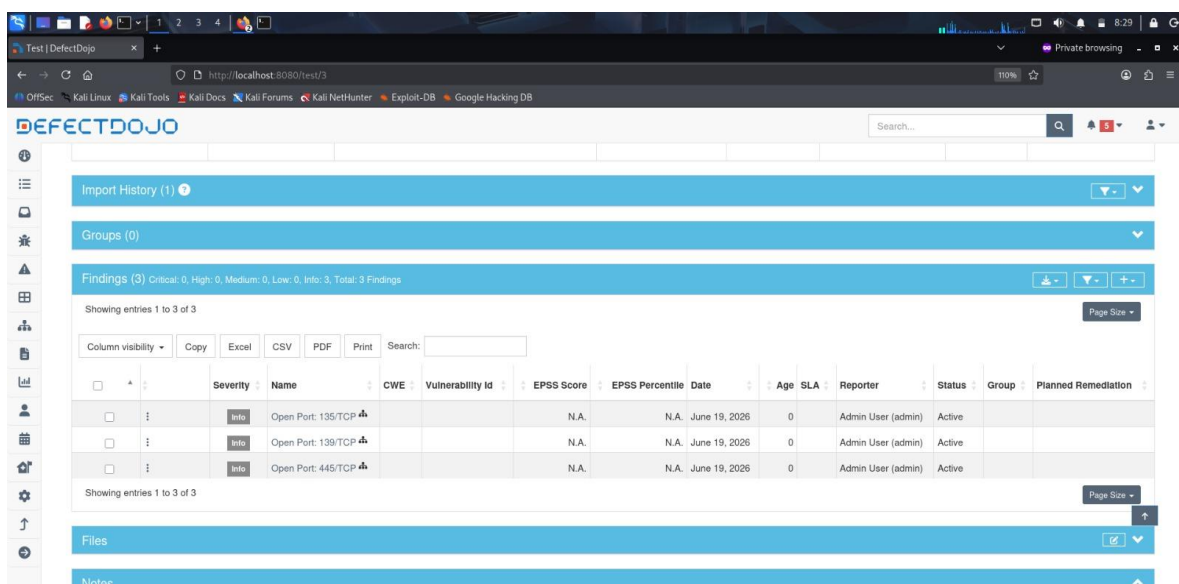
Import History (1)

Groups (0)

Findings (0) Critical: 0, High: 0, Medium: 0, Low: 0, Info: 0, Total: 0 Findings

Step-8: Specific Scan Findings View

Once a report with actionable data is populated, this view displays the immediate findings from that specific import. In this view, 3 open ports (135/TCP, 139/TCP, 445/TCP) have been successfully logged as **Info** severity.



DEFECTDOJO

Import History (1)

Groups (0)

Findings (3) Critical: 0, High: 0, Medium: 0, Low: 0, Info: 3, Total: 3 Findings

Showing entries 1 to 3 of 3

Column visibility Copy Excel CSV PDF Print Search:

Severity	Name	CWE	Vulnerability Id	EPSS Score	EPSS Percentile	Date	Age	SLA	Reporter	Status	Group	Planned Remediation
Info	Open Port: 135/TCP			N.A.	N.A.	June 19, 2026	0		Admin User (admin)	Active		
Info	Open Port: 139/TCP			N.A.	N.A.	June 19, 2026	0		Admin User (admin)	Active		
Info	Open Port: 445/TCP			N.A.	N.A.	June 19, 2026	0		Admin User (admin)	Active		

Showing entries 1 to 3 of 3

Files

Notes

Step-9: Main Product Open Findings Dashboard

This is the comprehensive dashboard displaying all active open findings for the product. It aggregates vulnerabilities from multiple scans, currently showing 5 total findings (including 1 **Medium** severity SMB vulnerability alongside the **Info** logs).

☐	Severity	Name	CWE	Vulnerability Id	EPSS Score	EPSS Percentile	Known Exploited	Used In Ransomware	Date Added to KEV	Date	Age	SLA	Reporter	Found By	Status	Group	Service	Planned Remediation	Reviewers
☐	Medium	SMB Service Detected on Port 445			N.A.	N.A.	No	No		June 19, 2026	0	90	Admin User (admin)	Pen Test	Active				
☐	Info	Open Port: 135/TCP			N.A.	N.A.	No	No		June 19, 2026	0		Admin User (admin)	Nmap Scan	Active				
☐	Info	Open Port: 139/TCP			N.A.	N.A.	No	No		June 19, 2026	0		Admin User (admin)	Nmap Scan	Active				
☐	Info	Open Port: 445/TCP			N.A.	N.A.	No	No		June 19, 2026	0		Admin User (admin)	Nmap Scan	Active				
☐	Info	Network Connection Analysis			N.A.	N.A.	No	No		June 19, 2026	0		Admin User (admin)	Pen Test	Active				

Conclusion

The SecureGov Framework provides an integrated approach to security monitoring, vulnerability assessment, and risk management by combining multiple open-source cybersecurity tools into a unified workflow. It enhances the efficiency of identifying, analyzing, and managing security risks while supporting governance and compliance objectives. Overall, the framework demonstrates a practical, scalable, and cost-effective solution for improving an organization's cybersecurity posture and enabling proactive risk management.

References

1. Offensive Security. (2025). Kali Linux Documentation.

<https://www.kali.org/docs/>

2. Nmap Project. (2025). Nmap Reference Guide.

<https://nmap.org/book/>

3. Greenbone Networks. (2025). Greenbone Community Edition (OpenVAS/GVM) Documentation.

<https://greenbone.github.io/docs/>

4. Wireshark Foundation. (2025). Wireshark User's Guide.

<https://www.wireshark.org/docs/>

5. Rapid7. (2025). Metasploit Documentation.

<https://docs.rapid7.com/metasploit/>

6. DefectDojo Project. (2025). DefectDojo Documentation.

<https://documentation.defectdojo.com/>

7. OWASP Foundation. (2025). OWASP Testing Guide.

<https://owasp.org/www-project-web-security-testing-guide/>

8. National Institute of Standards and Technology (NIST). (2024). Cybersecurity Framework (CSF) 2.0.

<https://www.nist.gov/cyberframework>